The King Hussein School for Computing Sciences
Department of Computer Science
Algorithms Design & Analysis - Spring 2022

Final Exam

Full Name: Reference Solution **Student ID:**

Instructions.

- Time allowed: 120 minutes.
- The exam is made of 9 pages. If you don't have a total of 9 pages, report this to the instructor.
- Calculators are allowed. All other electronic devices are not allowed.
- Write your answers clearly: you may not receive full credit if your answer is unreadable.

Question	Points	Score
1	16	
2	15	
3	35	
4	34	
Total		

Circle your instructor:

- ○ Dr. Ibrahim Albluwi
- ○ Dr. Khaled Mahmoud
- ○ Dr. Raghda Hriez

Tuesday 14/6/2022

Question 1: Miscellaneous (16 points)

1.A [6 pts] For each pair of functions below, check *all* the asymptotic relations that apply.

$f(n)$	$g(n)$	$f = O(g)$	$f = \Omega(g)$	$f = \Theta(g)$
n^3	$(n^2)^{\log n}$	✔		
$(2)^{n^2}$	2^{2n}		✔	
$2^{\log_4 n}$	$(n)^{(\log n)^2}$	✔		

1.B [6 pts] Solve each of the following recurrence equations using the Master Method. If the Master Method does not apply, explain briefly why and then solve the equation using iterative substitution or the recursion tree method.

- $T(n) = 9T(\frac{n}{3}) + n^{0.7}, \qquad T(1) = 1$

  Using the Master Method: $n^{0.7} = O(n^{\log_3 9-\epsilon}) = O(n^{2-\epsilon})$ for $0 < \epsilon \leq 1.3$
  Therefore, $T(n) = O(n^2)$

- $T(n) = T(n-1) + \log n, \quad T(1) = 1$

  The master method does not apply because the recurrence does not follow the form $T(n) = aT(\frac{n}{b}) + f(n)$

  Using The recursion tree method:
  $T(n) = \log(n) + T(n-1) = \log(n) + \log(n-1) + T(n-2)$
  $T(n) = \log(n) + \log(n-1) + \ldots + \log(n-k) + T(n-k-1)$

  We stop when k = n-2, i.e. when

  $T(n) = \log(n) + \log(n-1) + \ldots + \log(2) + T(1)$
  $T(n) = \log(n!) = \Theta(n \log n)$ using Sterling's Approximation.

1.C [4 pts] Pick the right sorting algorithm for each of the following scenarios.

- You are working on an embedded system (an ATM - نظام مدمج لصراف آلي) with a very limited memory and you want to sort `200000` transactions, which sorting algorithm will you use: *merge* sort or *heap* sort? Justify your answer.

  I will use Heapsort because it is an in-place sorting algorithm and its worst case asymptotic complexity is not worse than Mergesort.

- Which sorting algorithm would you use to sort file names in a file explorer: *quick* sort without shuffling or *merge* sort. Keep in mind that users frequently click on the sort option even if the files are already sorted or partially sorted? Justify your answer.

  I will use Mergesort because it is guaranteed to run in $\Theta(n \log n)$ in all cases, whereas Quicksort can run in $\Theta(n^2)$ if not shuffled.

Question 2: Greedy Algorithms (15 points)

2.A [6 pts] Suggest **two greedy strategies** for each of the following problems. You may suggest any greedy strategy, even if it does not always find the optimal solution.

- Fractional knapsack.

```
Strategy 1: Pick the lightest item first.
Strategy 2: Pick the heaviest item first.
```

- Activity selection problem.

```
Strategy 1: Pick the shortest activity first.
Strategy 2: Pick the longest activity first.
```

2.B [3 pts] In the 0/1 knapsack problem, does selecting the item with the minimum weight result in the optimal solution in all cases? If not, provide a counter example.

```
No it does not. For example, consider a knapsack of capacity 2kg and two
items: (1kg/$1) and (2kg/$2). Picking the lightest item gives a total
profit of $1, while picking the other item gives $2.
```

2.C [6 pts] Complete the following pseudo-code for solving the fractional knapsack problem.

```
// W is the capacity, v[] contains the values (profit) of the
// items, w[] contains the weights and n is the number of items.

FRACTIONAL_KNAPSACK(w[], v[], n, W)

Sort v[] and w[] based on v[i]/w[i] in decreasing order

load = 0, profit = 0, i = 1

while i <= n and load < W:
      if ( w[i] + load <= W )

            Take all of item i

            load += w[i]

            profit += v[i]

      else
            Take (W-load) of item i

            profit += (v[i]/w[i]) * (W-load)

            load = W
      i=i+1
```

Question 3: Dynamic Programming (35 points)

3.A Consider the following instance of the Longest Common Subsequence problem:

`X = ASDBF` `Y = SADF`

(I) `[8 pts]` Use the bottom-up dynamic programming algorithm studied in class to find the **length** of the longest common subsequence. Show the table.

`Length = 3`

		A	S	D	B	F
		0	0	0	0	0
S	**0**	0	1	1	1	1
A	0	**1**	**1**	1	1	1
D	0	1	1	**2**	**2**	2
F	0	1	1	2	2	**3**

(II) `[4 pts]` Find the longest common subsequence. Show your work on the table.

`LCS = ADF`

3.B `[2 pts]` Consider the following table for storing solutions for subproblems in the dynamic programming algorithm for the Longest Common Subsequence problem. Provide an example of a pair of strings `X` and `Y`, that could possibly lead the algorithm to produce the table.

`X = AACC`

`Y = CCDD`

		C	C	D	D
	0	0	0	0	0
A	0	0	0	0	0
A	0	0	0	0	0
C	0	1	1	1	1
C	0	1	2	2	2

3.C `[6 pts]` Consider three strings, `X`, `Y`, and `Z`. Assume that `LCS(i, j, k)` is the length of the longest common subsequence between the first `i`, `j` and `k` characters of `X`, `Y` and `Z` respectively. Provide a **recursive definition** for `LCS(i, j, k)` that captures the **optimal substructure** of the problem.
(Do not forget the base cases!)

```
                  ⎧ 0                                  if (i = 0 or j = 0 or k = 0)
                  ⎪
                  ⎪ 1 + LCS(i-1, j-1, k-1)             if (X[i] == Y[j] == Z[k])
                  ⎪
LCS(i, j, k)=     ⎪ max(LCS(i-1, j, k),                if (X[i] != Y[j] or
                  ⎪     LCS(i, j-1, k),                    X[i] != Z[k] or
                  ⎪     LCS(i, j, k-1))                    Y[j] != Z[k])
                  ⎩
```

3.D [6 pts] Consider the 0-1 Knapsack problem studied in class. Assume that our goal changes to maximizing the **total weight** of items in the knapsack instead of the total value (profit), but the other constraints remain the same.

If `OPT(i, j)` is the maximum sum of weights possible for items that fit in a knapsack of capacity `j`, considering the first `i` items only. Provide a **recursive definition** for `OPT(i, j)` that captures the **optimal substructure** of the problem. (Do not forget the base cases!)

(`w[i]` = weight of item `i`, `v[i]` = value (or profit) of item `i`, `W` = knapsack capacity, `N` = number of items)

```
                  ⎧ 0                                  if (i = 0 or j = 0)
                  ⎪
OPT(i, j) =       ⎨ OPT(i-1, j)                        if (w[i] > W)
                  ⎪
                  ⎩ max(w[i] + OPT(i-1, j-w[i],        otherwise
                             OPT(i-1, j))
```

3.E Answer the following questions about the bottom-up dynamic programming solution studied in class for the Matrix Chain Multiplication problem. Each question is independent of the other questions.

(I) [6 pts] Consider the following table storing the optimal split points for the chain $A_1A_2A_3A_4$. Use the table to find what the optimal parenthesization is.

$((A_1(A_2A_3))A_4)$

0	0	0	0	0
0	0	1	1	3
0	0	0	2	3
0	0	0	0	3
0	0	0	0	0

(II) [3 pts] Provide the dimensions of the matrices in a matrix chain $A_1A_2A_3A_4$ that can produce the following table storing the optimal multiplication cost.

All of dimensions 1x1

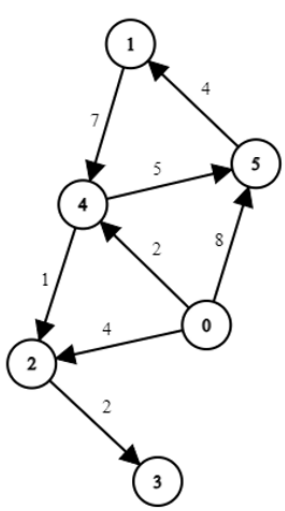
0	0	0	0	0
0	0	1	2	3
0	0	0	1	2
0	0	0	0	1
0	0	0	0	0

Question 4: Graphs (34 points)

4.A [8 pts] For each description on the left, write the letter of the **best**-matching asymptotic complexity. You may use each letter once, more than once, or not at all. (V = number of vertices, E = number of edges)

D	Checking if an edge ($u \to v$) exists in the adjacency list of a graph.	**A.** $O(1)$
A	Adding an edge ($u \to v$) to the adjacency list of a dense graph (without checking for parallel edges).	**B.** $\Theta(V)$
B	Finding the out-degree of a vertex u from an adjacency matrix.	**C.** $O(E+V)$
B	Finding the in-degree of a vertex u from an adjacency matrix.	**D.** $O(\deg(u))$
F	Checking if a graph represented as an adjacency matrix is symmetric.	**E.** $O(\deg(v))$
C	Checking if an undirected graph is connected.	**F.** $O(V^2)$
G	Running Dijkstra's shortest paths algorithm using a binary heap as a priority queue.	**G.** $O(E \log V)$
H	Running Dijkstra's shortest paths algorithm using a sorted array as a priority queue.	**H.** $O(EV)$

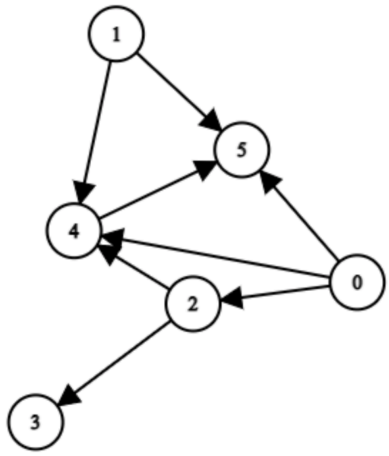
4.B [5 pts] Apply Dijkstra's shortest paths algorithm on the following graph (`0` is the source vertex). Draw a table showing how the distances and parent of each vertex changes at each step of the algorithm.



Vertex	distance	Parent
0	0	-1
1	∞ 11	5
2	~~∞~~ ~~4~~ 3	~~0~~ 4
3	∞ 5	2
4	∞ 2	0
5	~~∞~~ ~~8~~ 7	~~0~~ 4

4.C [3 pts] List the vertices of the following graph in topological order.

.



0 1 2 3 4 5

4.D Consider the following `Graph` class representing a simple, directed and weighted graph.

```
class Graph {
    int V; // No. of vertices
    list<int> *adj;

public:
    void add_edge(int u, int v);   // adds a directed edge u→v
    int weight(int u, int v);      // returns the weight of the edge u→v
    bool has_edge(int u, int v);   // does the edge u→v exist?

    stack<int> topological_sort(); // returns a stack containing the vertices
                                   // in topological order, where the vertex
                                   // at the top of the stack is the first
                                   // vertex in the topological order


    bool is_complete();
    void sssp(int source, int d[]);
};
```

You can assume that all the functions are already implemented, except functions `is_complete()` and `sssp(int source, int d[])`.

The questions are provided in the following page.

(I) [6 pts] Implement (in C++ or in pseudocode) function **`is_complete()`**, which returns `true` if the graph is complete and `false` otherwise.

```
bool is_complete() {
      int count = 0;
      for (int v = 0; v < V; v++)
            count += adj[v].size();
      return count == V * (V-1);
}
```

(II) [6 points] Solving the single-source shortest paths problem on a directed acyclic graph can be done correctly without Dijkstra's algorithm as follows:

For every vertex in the graph, **in topological order**, relax all of its outgoing edges.

Implement function **`sssp(int source, int d[])`** using this algorithm. The function receives the source vertex and an uninitialized array of size V for storing the shortest distances from the source to every vertex.

```
void sssp(int v, int d[]) {
      for (int w : adj[v])
            if (d[v] + weight(v, w) < d[w])
                  d[w] = d[v] + weight(v, w);
}

void sssp(int source, int d[]) {
      for (int i = 0; i < V; i++)
            d[i] = INT_MAX;
      d[source] = 0;

      stack<int> s = topological_sort();
      while (!s.empty()) {
            relax(s.top(), d);
            s.pop();
      }
}
```

4.E [6 pts] For each of the following problems, mention the name of the **closest problem** covered during the course, the name of the **best algorithm (or design technique)** you know for solving it, and the **running time** of the algorithm in the worst case.

(I) Assume you are a hacker who created a malware (برنامج تخريبي) capable of copying itself from one computer to another on a network. You want to attack a device that is part of a network. While you don't have access to the device you want to attack directly, you have access to another device on the same network and a diagram that describes how the computers are connected. In order to minimize the possibility of detection, your goal is to infect (تخرب) as few computers as possible before delivering the malware to the target device.

```
Closest Problem: Finding Shortest Paths on an unweighted graph.
Best Algorithm: BFT
Running Time: O(E+V), E = number of edges, V = number of vertices
```

(II) Assume you are a software engineer and want to test a program that consists of a number of functions. Knowing that the functions call each other, you don't want to test a function without first ensuring that all the functions that this function calls are working properly. What will you do to determine the sequence of functions to be tested?

```
Closest Problem: Topological Order
Best Algorithm: Reverse Post-order of DFT
Running Time: O(E+V), E = number of edges, V = number of vertices
```